



T.C.
SAKARYA ÜNİVERSİTESİ

BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
PROGRAMLAMA DİLLERİNİN PRENSİPLERİ ÖDEV RAPORU

C DİLİNDE NDP BENZETİMİ

B241210040 - Ahmet Faruk İKİZ

SAKARYA

Mayıs , 2026

Programlama Dillerinin Prensipleri Dersi

PDP 2.Ödev C Dilinde NDP Benzetimi

Ahmet Faruk İkiz^{b241210040 1A*}

Özet

Ödevde İstenenler:

Ödevde bir string dizisi şeklinde alınan 2 basamaklı sayıların kendisi, birler ve onlar basamağı gibi özelliklerinden her biri şehirdeki nüfus (kendisi), ilçe (onlar) ve mahalle (birler) sayısını temsil etmektedir. Bu sayılar parçalanarak, şu kurallar: “Mahalle sayısı, her ilçede eşit olacak şekilde dağılabilir. Toplam nüfus, her mahallede eşit olacak şekilde dağılabilir.” Sağlanacak şekilde hesaplanan sayılar sonucunda [java-faker](#) kütüphanesi yardımıyla önceden oluşturulmuş txt dosyalarından okunan değerlerle Şehir, İlçe, Mahalle, Kisi yapıları oluşturulacaktır. Şehir, İlçe, Mahalle yapıları Yerlesim yapısından kalıtım alıyor gibi yapılmalıdır.

Ardından başlangıçta kullanıcıdan alınan tur sayısı kadar simülasyon yürütülecektir. Her tur için kişiler 1 yıl yaşlanacaktır ve şehrin nüfusu “mevcutNüfus = (birler+onlar) * mevcutNüfus” kuralına göre artacaktır. Her tur sonuna gelindiğinde şehrin nüfusu 4 basamaklı olduysa bölünecektir. Şehirde birden çok ilçe varsa; bu ilçeler tam bölünüyorsa eşit, bölünmüyorsa fazlalık eski şehirde kalacak şekilde eski şehirdeki ilçeler yeni şehire mahalleleri ve kişileriyle beraber aktarılmalıdır. İlk ödevden farklı olarak bu modüler yapının C içerisinde kurulması ve nesne yönelimli benzetim yapılması istenmiştir.

Ödevde Uyguladığım Yaklaşım:

Ödevde C dilinde de aynı modülerlikten yararlanabilmek adına önceki ödevde kullandığım yardımcı servis ve arac sınıflarını da birer yapı (struct) olarak yazdım. Sadece model yapıları değil süreci yöneten modüller de yapı şeklinde kurgulanmasıyla javada yazdığım koda çok yakın bir proje elde ettim.

© 2026 Sakarya Üniversitesi.

Bu rapor benim özgün çalışmamdır. Faydalanmış olduğum kaynakları içerisinde belirttim. Herhangi bir kopya işleminde sorumluluk bana aittir.

Anahtar Kelimeler: C Language, OOP Simulation in C, Concepts of Programming Languages, Memory Management

1. GELİŞTİRİLEN YAZILIM

1.1 Modülerlik

Ödevde include ve src klasörlerinin altında; araçlar, main, modeller, motor, servisler ve SahteVeri olmak üzere 6 adet klasör kullandım:

- Araclar: karışık hesaplama fonksiyonları ve kodları servis yapılarından ayrı tutan yapıları barındırır.
- Main: main.c’yi barındırır.
- Motor: Ana mantığın yürütüldüğü Oyun yapısı bulunur.
- Modeller: İçerisinde veri tutan ve kendi iç verilerini yöneten model yapılarını barındırır.
- Servisler: Oyun yapısının görev yükünü hafifleterek modülerlik sağlayan alt yönetim yapıları.
- SahteVeri: Sahte veri üretiminde kullanılacak fakerla oluşturulmuş verileri içeren txt dosyaları.

1.2 Programın genel akışı ve Yapılar

1.2.1 Main dosyası:

SahteVeriUretici’nin örneği ilk ve tek kez üretilir ve main sonunda temizlenir. BaslangicGirdiIslemleri yapısının metotlarıyla tur sayısı ve sayı dizisini alıp Oyun yapısına gönderir. Oyun yapısının baslat() fonksiyon göstericisini çağırır.

1.2.2 Yapılar:

Header dosyasında bulunan her bir fonksiyona ek olarak yapıların içerisine fonksiyon göstericisi vardır. Bu göstericiler new_ fonksiyonunun içerisinde yapı göstericisine bellekte yer açılması sonrası ilgili fonksiyonlara bağlanarak yapı'ya aitlermiş gibi dışarıya sunulur. Bu sayede yapıların mesaj taşınması sağlanır. Fakat java gibi gerçek ndp içeren dillerde özellikle içerisinde durum tutan ve static olmayan her metotta gizli bir parametresi bulunur. Bunu c'de karşılamak için static olarak tasarlamadığım her metotta ilk parametre olarak yapının kendisini alacak şekilde tasarladım. Private metodları ise header dosyasına yazmayıp source dosyasında tuttum. İstisna olarak SahteVeriUretici yapısını her kisi örneklenirken tekrar tekrar oluşmaması ve dosyayı okumaması için program başında 1 kez bellekte oluşacak ve hep aynı adres üzerinden erişilecek şekilde yazdım.

1.2.3 Sahte veri ve dosya okuma:

SahteVeriUretici yapısı her seferinde dosyadan okumak yerine programın başında bir kez oluşur ve bir kez dosyayı okuyarak verileri char* dizilerine kaydeder. Gerektiğinde ramde duran bu dizilerden rastgele veri çekerek model yapılarına verir.

1.2.4 Genel akış:

Oyun yapısı mainden aldığı başlangıç parametrelerini OyunBaslatıcıServis yapısının YerleskeOlustur metoduna gönderir ve içi dolu şehirler dizisini alır. Burada OyunBaslatıcıServis tek tek for döngüleriyle model yapılarını üretir ve kurucularını çağırır. Kurucularında SahteVeriUretici ile dosyadan okunan sahte verilerin ataması yapılır. Ardından oyun yapısı ana loop içerisinde tur işlemlerini yönetir. Bölünme, nüfus artımı gibi süreçleri doğrudan yönetmez şehir yapısına haber verir şehir yapısı üstten aşağıya durumu bildirirerek hiyerarşik bir bölünme/nüfusArtımı sağlar. Tur işlemleri bitince private oyunSonuSatırSutunSor() metodunu çağırır bu metod kullanıcıdan satır sütun girdisi alıp yazdırıcıServis'in detayYazdır() metoduna iletir. Bu metod yine hiyerarşik olarak model yapılarının ekranaYazdır() metodunu çağırır.

1.3 Algoritmalar:

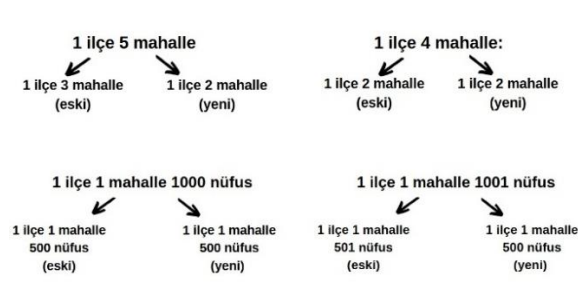
1.3.1 Bölünme algoritması:

Programda bir şehrin bölünmesinde şu 6 durum gözlenebilir:

Çift Sayıda İlçe, Tek Sayıda İlçe, 1 ilçe çift sayıda mahalle, 1 ilçe tek sayıda mahalle, 1 ilçe 1 mahalle çift nüfus, 1 ilçe 1 mahalle tek nüfus.

Çift ve tek sayı durumları, tek sayı durumunda 1 eksik olarak aktarılacağından tam sayı bölmesi kullanılarak rahatlıkla çözülebilir. aktarılacakSayı = BirimSayısı / 2; (Tek ise 1 eksik hesaplar küsüratı atar)

1 ilçe olduğu durumlarda ise ayrıştırma Şekil 1'deki gibi olacaktır:



Şekil 1. 1 İlçe Bölünme Örneği

Burada ilk akla gelen çözüm iç içe iki if else bloğu ile tüm koşulları tek tek kontrol etmektir. Fakat burada farkedileceği üzere aslında her durumda benzer bir mantık alt birime geçirilerek uygulanmaktadır.

bolun() metodunun çalışma mantığı:

- 1.Yeni birim oluşturun.
- 2.Alt birim çok sayıdaysa alt birimi 2'ye böl yeni birime aktar. Tek sayıdaysa yeni alt birim oluşturun bir alt birimin bolun() metodunu çağır dönen değeri yeni alt birime ata, yeni alt birimi yeni birime ekle.
3. Yeni birimin ve eski birimin nüfusGuncelle() metodunu çağır
4. birimi return et.

Modellerin içinde bulunan bu mantığa sahip hiyerarşik bolun() metodları sayesinde Oyun yapısı temiz ve okunabilir kalır. Program test edilebilir ve bakımı yapılabilir tutulur.

1.3.2 Nüfus artış Algoritması:

nufusArttir ve nufusGuncelle metodu: Şehir yapısından başlayarak her model yapısı bir alt yapının nufusArttir metodunu çağırıp artisOrani değişkenini aktarır. En son mahalle nüfusunu artırır ve kendi nüfusunu return eder. Üst yapılar alt yapıların nüfuslarını toplayarak kendi nüfusunu hesaplar ve bir üste iletir. Aşağıdan yukarıya aktarımla şehrin nüfusu hem artmış hem de tekrar hesaplanmış olur nufusGuncelle de benzer mantıkla çalışır.

2. ÇIKTILAR:

(Şehrin tüm bilgilerinin yazdırılması çok uzun sürdüğünden Şekil 2. ve 3.'de ilk 5 kişi koyulmuştur. Ayrıca tur sayısı fazla tutulursa nüfuslar çok artacağından konsol ekranına sığmayacak ve analiz yapması zorlaşacaktır.)

Girdi 1: turSayisi: 3, sayilarString: 18 25 79 37 62 86 17 50

```
Oyun Sonu:
[3744]-[810]-[11858]-[3375]-[31944]
[5632]-[4212]-[5500]-[12320]-[5632]
[3744]-[11858]-[5400]-[31944]-[5632]
[551]-[3080]-[5632]-[3744]-[11858]
[3375]-[15972]-[5632]-[3159]-[2750]
[12320]-[5632]-[3744]-[11858]-[2700]
[15972]-[5632]-[551]-[3080]-[5632]

Satir Girin (0'dan baslar):
0
Sutun Girin (0'dan baslar):
1
Sehir: Nevada-Nufus: 810
Ilce: Quigleybury-Nufus: 405
Mahalle: Schinner Corner-Nufus: 135
Kisiler:
124-Stephani Schumm-45
125-Alfred Simonis-5
126-Trula Nader-32
127-Chance Turcotte-51
128-Greg Schultz-24

139-Gregorio Keebler-45
140-Amelia Witting-19
141-Jewel Ryan-48
142-Jeremy Romaguera-15
143-Catherine Shields-34
Ilce: West Brandy-Nufus: 405
Mahalle: Jayme Circle-Nufus: 135
Kisiler:
```

Şekil 2. Girdi 1 için program çıktısı

Şekil 2.'de görüldüğü üzere Şehir sayısı başlangıçta girilen sayıların sayısından fazladır. Nüfusu birebir aynı olan örneğin 5632, 11858 gibi nüfuslardan da bunların bölünen şehirler oldukları anlaşılmaktadır. Yaşlara bakıldığında normalde 0-50 arası atanan yaşlar arasında 51 yaşında Chance Turcotte kişisi tur geçtikçe gerçekten yaşlarının arttığını kanıtlamaktadır. Nevada Şehrinin nüfusuna ve ilçe ve mahallelerinin nüfuslarına bakıldığında gerçekten eşit dağıldığı görülmektedir. ($810 = 405 + 405$ | $810 = 135 * 6$) başlangıçta 25 şeklinde verilen sayı kurallara göre 26 olmuş bu durumda 2 ilçe ve 6 mahalle şeklinde oluşturulmuştur ve çıktılarla uyumaktadır.

Girdi 2: turSayisi: 3, sayilarString: 77 11 89 91 83 58 39 12 83 41 99

```
Oyun Sonu:
[2156]-[88]-[1408]-[5346]-[1408]
[1650]-[1350]-[324]-[1408]-[1232]
[5346]-[2464]-[1408]-[4356]-[1408]
[1408]-[4356]-[2156]-[1408]-[3564]
[1408]-[1100]-[675]-[1408]-[1232]
[3564]-[1232]-[1408]-[4356]-[1408]
[1408]-[4356]

Sehir: Utah-Nufus: 324
Ilce: South Santanamouth-Nufus: 324
Mahalle: Bergstrom Viaduct-Nufus: 162
Kisiler:
563-Eilene Schaden-14
564-Brady Rath-45
565-Roman Kilback-41
566-Shara Gaylord-12
567-Dana McKenzie-42
568-Arvilla Schaden-42

569-Joye Hegmann-51
570-Rocco Wehner-19
571-Delmer West-16
572-Mauro Johnston-5
573-Anthony Boyer-3
574-Curtis Adams-47
Mahalle: Cormier Mews-Nufus: 162
Kisiler:
```

Şekil 3. Girdi 2 için program çıktısı

11 -> 1+1 artış oranı = 2. $2*11 = 22$ yeni nüfus. 22-> 2+2 artış oranı 4. $4*22 = 88$. 0.satır 1.sütunda 88 sayısını görebiliriz. Ayrıca 1.satır 1.sütun için başlangıçta 39 sayısıdır. 3 ilçe her ilçede 3 mahalle şeklinde üretilip nüfusu 45 sayısına çıkarılmıştır. $4+5 = 9$. $45*9 = 405$. $0+5 = 5$. $405 * 5 = 2025$. 2025 4 basamaklı başta 3 ilçemiz vardı ve her mahallede 225 kişi var. Eski Şehrin yeni nüfusu: $225 * 6 = 1350$. Yeni şehrin nüfusu: $225 * 3 = 675$ (4.satır 2.sütun). 1.satır 2.sütun için ise henüz bölünme noktasına ulaşamamıştır 1 ilçesi 2 mahallesi vardır.

Şehirler hızla büyüyüp bölünmektedir ve gitdikçe az ilçe ve mahalleli yüksek nüfuslu şehirler görülmeye başlanacaktır.

3. SONUÇ:

Bu ödevde gerçek ndp destekleyen dillerde kurucu metot, yıkıcı metot this parametresi, kalıtım, soyutlama gibi kavramları derleyicinin nasıl yönettiğini bizzat kendim yazarak deneyimledim. Ndp desteklemeyen bir dilde yaşadığım zorluklarla Ndp'nin mantığını ve önemini daha iyi anladım. Ayrıca c'de dinamik, heap bellek bölgesinde malloc, realloc gibi fonksiyonlarla yer tahsis edip tekrar o yeri genişletme işlemleri yaparak c'de bellek yönetimi pratiği yapmış oldum. C'de stringlerin olmaması sebebiyle string literal, char* char[] gibi birbirine benzer ama farklı kavramları öğrendim stringleri daha iyi anlamış oldum. <string.h> kütüphanesindeki pek çok fonksiyonun mantığını anladım.